UQAR

FONDAMENTAUX DE L'APPRENTISSAGE AUTOMATIQUE

8INF867 (MS)

UNIVERSITÉ DU QUÉBEC À RIMOUSKI

DÉPARTEMENT DE MATHÉMATIQUES, D'INFORMATIQUE ET DE GÉNIE

RAPPORT

Travail pratique 1

ÉTUDIANT(E) :
Pier-Luc Dupéré

PROFESSEUR :
Yacine Yaddaden, Ph. D.

Date : 22 octobre 2026

Table des matières

1 Introduction **3**

2 Choix, exploration et compréhension des données **3**

3 Prétraitement et représentation des données **4**

4 Sélection / réduction de dimension **5**

5 Compréhension et recherche sur l'algorithme **6**

6 Protocole de validation et métriques **8**

7 Présentation et interprétation des résultats **10**

8 Bilan critique de l'algorithme **11**

1 Introduction

pld

2 Choix, exploration et compréhension des données

Le projet OASIS[] met gratuitement à la disposition des chercheurs des données d'imagerie du cerveau. On a utiliser la base de données OASIS 1 qui contient des images à résonance magnétique ayant été diagnostiquer pas des médecins professionnels. Dans cette base de données, on retrouve le résultats des patients et qui sont classifier en 4 catégories. De plus, la base de données contient d'autres informations et évaluations, comme les résultats de tests cognitifs et l'âge des patients etc.

3 Prétraitement et représentation des données

En ce qui concerne la préparation des données, on a extrait des coupes axial du cerveau de chaque patients. Pour faire la sélection des tranches, on a utiliser un logiciel appeler XMedCon[]. On a pris 25 coupes par échantillon. Les coupes de 80 à 104 inclusivement de chaque fichier hdr.

Dans ce travail, seulement 15 échantillons pour chacune des trois catégories :

— Démence légère (Mild Demented)
— Sans démence (Non Demented)
— Démence très légère (Very Mild Demented)

Le code pré-traite une image IRM du cerveau. Il la convertit en niveaux de gris, applique un flou gaussien, puis utilise le seuillage d :Otsu et des opérations morphologiques pour nettoyage de l'image. Il repère ensuite les contours situés dans la zone centrale afin de créer un masque du cerveau. Ce masque est appliqué à l'image, puis la zone obtenue est recadrée et redimensionnée en 128×128 pixels pour faciliter son analyse. On applique cela a tout les IRM et on sauvegarde le tout sous forme de png.

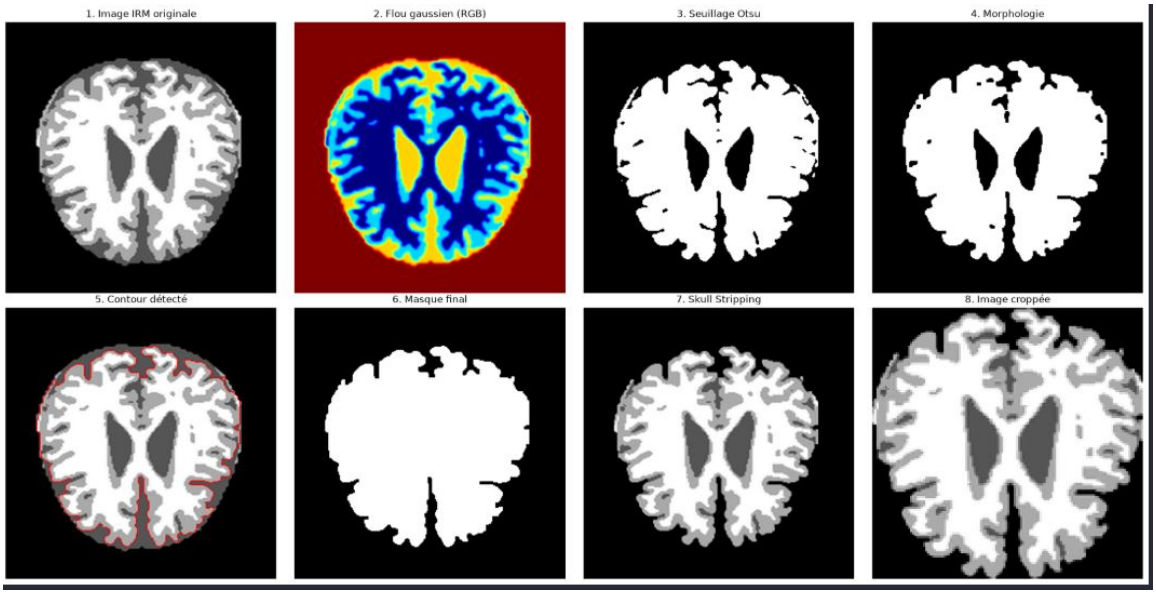


FIGURE 1 – Étapes du prétraitement des images IRM

4 Sélection / réduction de dimension

Le Local Binary Pattern (LBP) de la librairie scikit-image[] est utilisé afin de réduire et sélectionner l'information des IRM. Cette fonction compare les pixels voisins au pixel central. Cela forment un motif binaire qui représente la texture autour du pixel. On peut ensuite utiliser les motifs obtenus pour générer un histogramme pour caractériser et classer les textures. Dans notre étude, la classification des textures revient à classer les échantillons selon leur catégorie.

Les parameters de la fonction sont les suivants :
- P=8 (nombre de point)
- R=3 (rayon)
- method="default" (changements d'intensité en niveaux de gris sans rotations.)

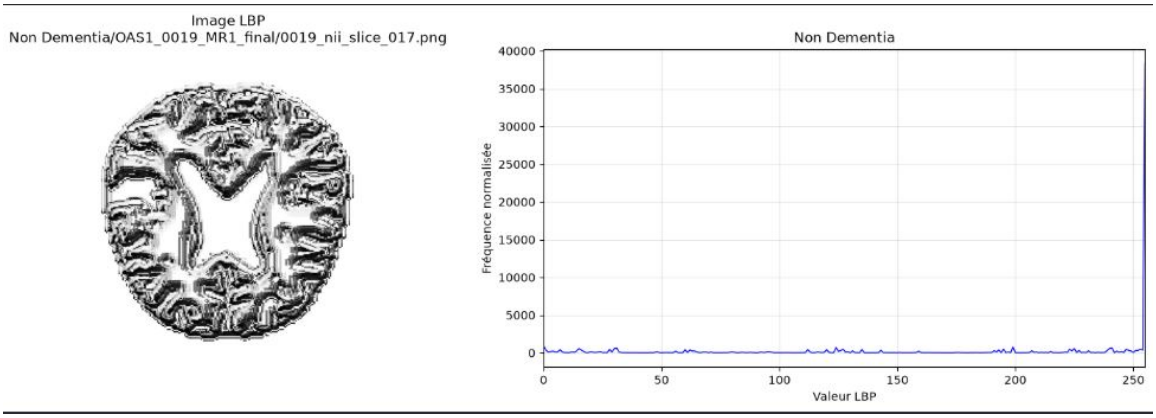


FIGURE 2 – À gauche : représentation de l'image LBP ; à droite : histogramme correspondant

De plus, une réduction de dimension par analyse en composantes principales de type PCA est appliquée aux données dans le pipeline[] d'execution.
Les parameters du PCA :
- `n_components=0.95` (% de la variance des données conservée)
- `svd_solver=auto` (méthode de calcul utilisé)
- `power_iteration_normalizer=auto`

5 Compréhension et recherche sur l'algorithme

```python
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

# === Pipeline SVM  LAISSEZ PAR DEFAULT ===
# https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html
svm_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("PCA", PCA(
        n_components=0.95,
        svd_solver="auto",
        power_iteration_normalizer="auto",
    )),
    ("svm", SVC(
        kernel="linear",
        C=10,
        gamma="scale",
        coef0=0.0,
    ))
])
```

FIGURE 3 – Pipeline réalise une classification avec un SVM

Dans l'image ci-dessus, on peut voir un pipeline qui standardise les caractéristiques et puis qui réduit leur nombre en conservant 95 % de la variance. Par la suite, on utilise un SVM pour classer les échantillons.
Dans le tableau ci-dessous, il y a un résumé synthèse de tout les principale hyperparamètres. On peut voir la fonction que chacun de chacun ainsi que leur effets sur l'hyperplan qui segmente les échantillons. De plus, les valeurs typique qui vont être tester plus tard.

TABLE 1 – Principaux hyperparamètres d'un SVM []

Hyperparamètre	**Fonction**	**Effets**	**Valeurs**
`kernel`	Type de noyau utilisé	Détermine la forme de la frontière de décision	`linear`, `poly`, `rbf`, `sigmoid`
`C`	Pénalisation des erreurs de classification	Une faible valeur favorise une marge plus large et peut entraîner du sous-apprentissage ; une valeur élevée pénalise davantage les erreurs et peut favoriser le sur-apprentissage	0.01 à 100
`gamma`	Inverse du rayon d'influence, notamment pour le noyau RBF	Une faible valeur produit une frontière plus lisse ; une valeur élevée peut la rendre plus complexe	0.001 à 10.0
`degree`	Degré du polynôme utilisé par le noyau polynomial	Un degré plus élevé permet une frontière de décision plus flexible	2 à 5
`coef0`	Terme constant des noyaux polynomial et sigmoid	Influence la forme du noyau, notamment dans le cas du noyau polynomial	0.0 à 1.0

6 Protocole de validation et métriques

Afin de cherche la meilleure combinaison de paramètres pour un modèle. On a utiliser la librairie GridSearchCV[]. Elle teste les combinaisons indiquées et évalue chacune avec une validation croisée, puis sélectionne celle qui obtient le meilleur score.

```
        "svm__gamma": ["scale", "auto"],
        # "svm__gamma": [0.1, 0.5, 0.75, 0.9, 1, 1.1, 1.5, 2, 10],
        "svm__degree": [2, 3, 4, 5],
        "svm__coef0": [0, 0.1, 0.25, 0.5, 0.75, 1],
    },
    {
        "PCA__n_components": [0.80, 0.85, 0.90, 0.95, 0.99],
        "svm__kernel": ["sigmoid"],
        "svm__C": [0.1, 0.5, 0.75, 0.9, 1, 1.1, 1.5, 2, 10],
        "svm__gamma": ["scale", "auto"],
        # "svm__gamma": [0.1, 0.5, 0.75, 0.9, 1, 1.1, 1.5, 2, 10],
        "svm__coef0": [0, 0.1, 0.25, 0.5, 0.75, 1],
    },
]

grid_search = GridSearchCV(
    estimator=svm_pipeline,
    param_grid=param_grid,
    scoring="accuracy",
    cv=cv,
    n_jobs=-1,
    refit=True,
    verbose=0
)

grid_search.fit(x_train, y_train)
best_svm_pipeline = grid_search.best_estimator_
```

FIGURE 4 – Utilisation de GridSearchCV

Pour chaque combinaison, GridSearchCV évalue les performances en validation croisée (cv), avec l'accuracy comme mesure. Il teste chacune des combinaisons répétée pour les différentes validations croisées avec StratifiedKFold[]. Cela signifie que chaque combinaison de paramètres sera évaluée sur ces 10 découpages, puis comparée selon son accuracy moyenne.
Après l'entrainment du model, on prend le meilleur résultat avec les meilleurs para-

mètres.
Pour finir, le meilleur modèle est évalué sur les données de test, qu'il n'a pas vues pendant son entraînement. Il prédit la classe de chaque échantillon. Ces prédictions sont comparées aux vraies classes pour calculer son exactitude et produire des documents afin de faciliter l'analyse des performances du modèle sur les données de test.

7 Présentation et interprétation des résultats

8 Bilan critique de l'algorithme

Les méthodes à noyaux offrent une grande flexibilité et permettent de modéliser des relations non linéaires, même avec des jeux de données de taille modérée. Elles reposent aussi sur des bases théoriques solides et peuvent être adaptées à différents types de données.
Elles présentent toutefois des limites : leur coût de calcul augmente fortement avec le nombre d'échantillons, leurs performances dépendent du réglage des hyperparamètres et leurs décisions sont parfois difficiles à interpréter. La prédiction peut aussi être lente, et l'obtention de probabilités nécessite une étape de calibrage supplémentaire.
Les SVM peuvent souffrir de surapprentissage lorsque leur modèle est trop complexe, ou de sous-apprentissage lorsqu'il est trop simple. Le réglage des hyperparamètres et la validation croisée permettent de trouver un bon équilibre et d'améliorer leur capacité à généraliser à de nouvelles données.
[1]

Références

[1] Aurélien Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. "O'Reilly Media, Inc.", 2019. pages 11